

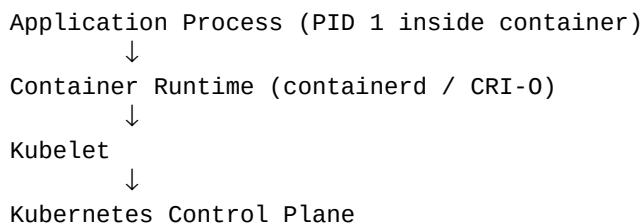
Kubernetes Exit Codes, Signals, Evictions, Lifecycle Hooks & Graceful Shutdown

A Production-Oriented Deep Dive for Long-Term Revision

This document summarizes core Kubernetes runtime behavior around Linux signals, container exit codes, OOM kills, evictions, graceful shutdown, lifecycle hooks, endpoint removal, and production-grade rolling update behavior. The goal is not just memorization, but building the correct mental model of how Kubernetes, container runtimes, Linux, networking, and applications interact during startup and shutdown.

1. The Fundamental Mental Model

Kubernetes does not directly kill your application. Kubernetes orchestrates containers through the kubelet and container runtime, while Linux ultimately manages the processes and signals.



Most Kubernetes debugging eventually becomes Linux process debugging. Understanding signals, cgroups, and process lifecycle is more important than memorizing YAML.

2. Linux Signals

Signals are asynchronous notifications sent to Linux processes.

Signal	Number	Purpose
SIGTERM	15	Graceful shutdown request
SIGKILL	9	Immediate forced kill
SIGINT	2	Ctrl+C interrupt
SIGSEGV	11	Segmentation fault

SIGTERM does NOT immediately kill a process. It politely asks the process to shutdown gracefully. The application decides how to react.

3. Exit Codes

Processes return exit codes when they terminate.

Exit Code	Meaning
0	Successful exit
1	Generic failure
126	Permission issue
127	Command not found
137	SIGKILL
139	Segmentation fault
143	SIGTERM

4. Why 128 + Signal Number?

Unix systems historically use the convention:

Exit Code = 128 + Signal Number

Examples:

SIGKILL = 9 → 128 + 9 = 137

SIGTERM = 15 → 128 + 15 = 143

SIGSEGV = 11 → 128 + 11 = 139

This helps distinguish normal application exits from signal-based process termination.

5. Exit Code 137 — The Most Important One

Exit code 137 means the process died due to SIGKILL. However, SIGKILL itself may happen for multiple reasons.

Scenario	Meaning
Container OOMKill	Container exceeded cgroup memory limit
Node-level OOM	Entire node exhausted memory
Grace Period Expired	Application ignored SIGTERM
Manual kill -9	Operator or system forcibly killed process

Important production truth: Exit code 137 does NOT automatically mean OOMKilled.

6. cgroup OOM vs Node OOM

Containers run inside Linux cgroups with memory limits.

```
resources:  
  limits:
```

memory: "512Mi"

If the container exceeds 512Mi, Linux kills the process even if the node itself still has free memory.

This is called a cgroup OOM kill.

Node-level OOM happens when the entire machine runs out of memory. The Linux kernel then selects victim processes across the whole node.

7. Kubernetes Evictions

Eviction is different from OOMKill.

OOMKill	Eviction
Kernel kills process	Kubelet removes pod
Memory limit violation	Node pressure handling
Container-focused	Node-focused

Evictions happen during:

- Memory pressure
- Disk pressure
- PID exhaustion

8. QoS Classes

QoS Class	Eviction Priority
BestEffort	First to be evicted
Burstable	Medium priority
Guaranteed	Last to be evicted

9. Graceful Shutdown Flow

The Kubernetes graceful shutdown sequence is extremely important.

```
kubectl delete pod
↓
Pod enters Terminating
↓
Readiness becomes False
↓
Pod removed from Endpoints
↓
```

```
preStop hook runs
↓
SIGTERM sent
↓
Application performs graceful shutdown
↓
terminationGracePeriodSeconds countdown
↓
SIGKILL if still alive
```

This sequence prevents new traffic while allowing existing work to finish.

10. Endpoint Removal vs SIGTERM

These are two completely different responsibilities.

Mechanism	Responsibility
Endpoint Removal	Stop NEW traffic routing
SIGTERM	Tell application to shutdown gracefully

Endpoint removal happens BEFORE SIGTERM.

This is intentional because Kubernetes wants to stop NEW traffic before beginning shutdown.

11. What SIGTERM Actually Does

SIGTERM does not kill the application immediately.

It tells the application:

Please begin graceful shutdown now.

The application must then:

- Stop accepting new work
- Finish inflight requests
- Flush logs and telemetry
- Close DB connections
- Commit Kafka offsets
- Exit cleanly

12. terminationGracePeriodSeconds

```
terminationGracePeriodSeconds: 40
```

This defines the TOTAL shutdown budget before Kubernetes forcefully sends SIGKILL.

Important: preStop hook time is INCLUDED inside this total budget.

13. preStop Hook

preStop runs BEFORE SIGTERM.

```
lifecycle:
  preStop:
    exec:
      command: ["sleep", "10"]
```

This delays SIGTERM by 10 seconds.

The purpose is not to process requests. The purpose is to allow the networking ecosystem to react to endpoint removal.

14. Why preStop Exists

Distributed systems are eventually consistent.

Even after endpoint removal:

- Ingress controllers may still route briefly
- Cloud load balancers may still have cached targets
- Keepalive TCP connections may remain open
- HTTP/2 and gRPC streams may still exist

preStop gives these systems time to converge before shutdown begins.

15. Real Production Example — Rolling Updates

Suppose a Spring Boot API is processing payment requests.

Without graceful shutdown:

```
Endpoint removed
↓
SIGTERM immediately
↓
```

Ingress still routing briefly
↓
Requests hit half-shutdown app
↓
502 / connection reset

With preStop + graceful shutdown:

Endpoint removed
↓
preStop sleep 10
↓
Ingress/LB updates propagate
↓
SIGTERM
↓
App drains inflight requests
↓
Clean deployment

16. Existing Requests vs New Requests

Traffic Type	Behavior During Shutdown
New Requests	Should stop after endpoint removal
Existing Requests	May continue during graceful shutdown

The application itself must handle inflight requests. Kubernetes cannot finish business logic for the application.

17. What Happens If Application Ignores SIGTERM

If the application does not handle SIGTERM:

SIGTERM sent
↓
Application ignores it
↓
Grace period expires
↓
SIGKILL
↓
Requests abruptly die

This is why proper application shutdown handling is critical.

18. postStart Hook

Kubernetes lifecycle hooks include:

Hook	Purpose
------	---------

postStart	Run logic after container starts
preStop	Run logic before SIGTERM

postStart is commonly used for:

- Initialization
- Cache warming
- Dynamic configuration fetch
- Sidecar coordination

19. Probes vs Lifecycle Hooks

Feature	Purpose
startupProbe	Delay probe checks until startup complete
readinessProbe	Control traffic eligibility
livenessProbe	Restart unhealthy containers
Lifecycle Hooks	Run actions during transitions

20. PID 1 Problem

Inside containers, PID 1 has special signal behavior.

Bad example:

```
CMD ["sh", "start.sh"]
```

Signals may not propagate properly to the real application process.

Better:

```
ENTRYPOINT ["java", "-jar", "app.jar"]
```

Or use init systems like tini/dumb-init.

21. Common Real Production Use Cases for preStop

- Traffic draining during rolling updates
- AWS ALB/NLB deregistration propagation
- gRPC and HTTP2 connection draining

- Kafka consumer shutdown coordination
- WebSocket session draining
- Service mesh sidecar coordination
- Leader election handoff

22. Common Anti-Patterns

- Assuming endpoint removal instantly stops all traffic
- Assuming SIGTERM immediately kills the app
- Blindly adding sleep 300 in preStop
- Ignoring SIGTERM handling in applications
- Confusing Exit 137 with only OOM

23. Production Debugging Commands

```
kubectl describe pod <pod>
kubectl logs <pod> --previous
kubectl get events --sort-by=.metadata.creationTimestamp
kubectl describe node <node>
```

24. Final Mental Model

Endpoint Removal:
Stop NEW routing

preStop:
Delay shutdown start / allow propagation

SIGTERM:
Ask app to shutdown gracefully

Application:
Finish inflight work

Grace Period:
Maximum allowed shutdown time

SIGKILL:
Forced immediate death

25. Most Important Production Lessons

- Exit 137 means SIGKILL, not automatically OOM

- Graceful shutdown is mainly an application responsibility
- Endpoint removal and SIGTERM solve different problems
- preStop is mainly a coordination/draining mechanism
- Distributed networking updates are eventually consistent
- Kubernetes cannot finish business logic for your application
- Production reliability depends heavily on shutdown behavior

This document is intended to provide a long-term conceptual understanding rather than isolated memorization.

Mastering these concepts dramatically improves real-world Kubernetes debugging and production operations skills.